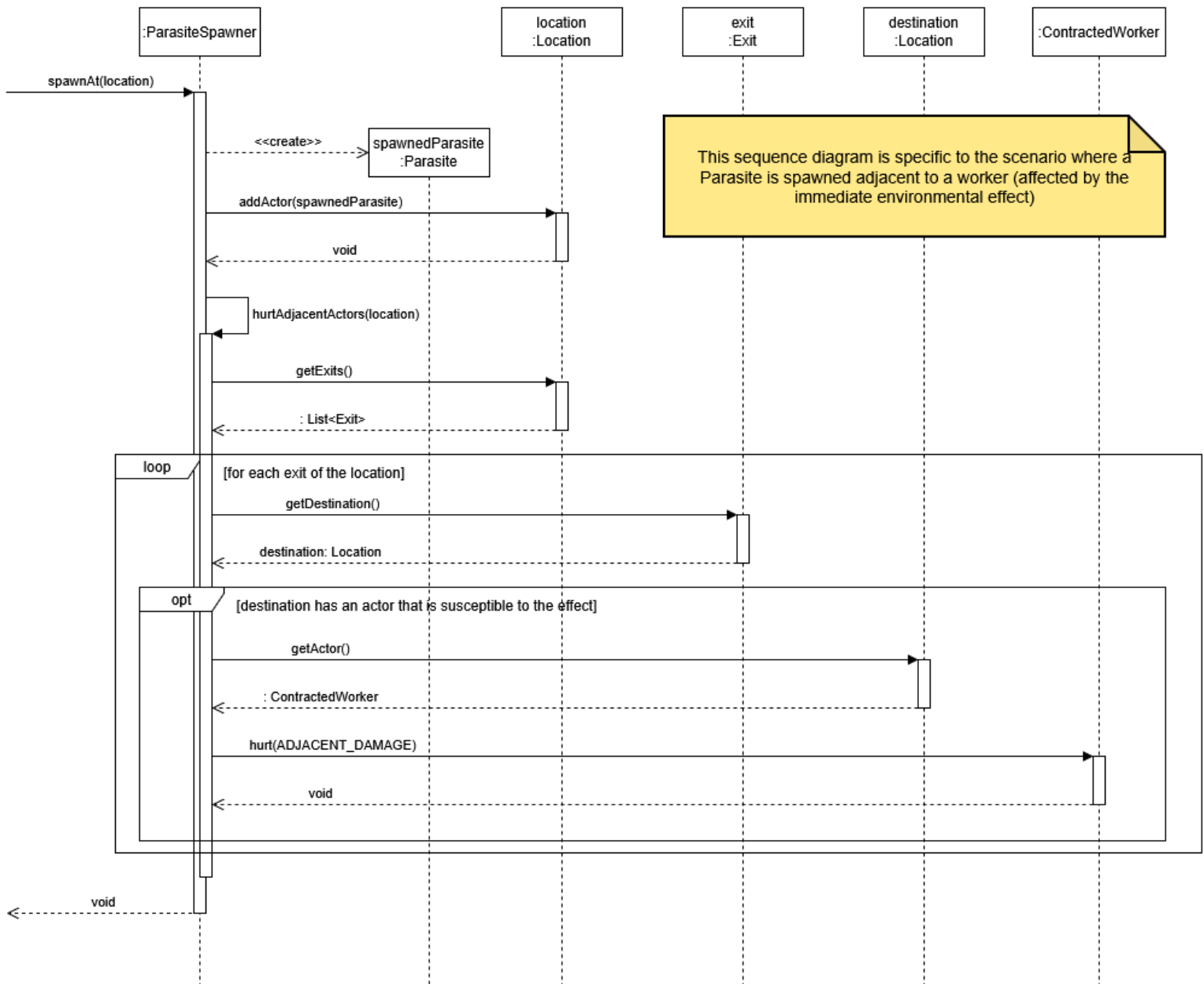


The UML sequence diagram for REQ4 is as follows:



This design rationale aims to cover the major decisions I've made across Requirement 4.

Decision 1a: Multiple targets need to be able to be infected by the parasite, each with different effects.

Chosen design: Create an interface Infectable and let each concrete class that can be infected implement them. Each of those classes themselves will define what effects they have when infected in the infection method, which is called by InfectStatus each turn.

Advantages	Disadvantages
If more entities are added that can be infected, they just need to implement Infectable and provide the logic for how they react to the infection. No need to modify any other classes.	Adding an interface would make the code more complex.
Each concrete class implementing Infectable have very different effects to each other.	

Alternative design: No Infectable interface. Instead, in InfectStatus, there will be a switch statement. Every turn will take the infected target (GameEntity), use instanceof and downcasting to check if the target is a certain concrete class, then define those effects for each case.

Advantages	Disadvantages
Code seems simpler without the interface, and every effect related to the infection is kept within one class.	Adding a new entity that can be infected will require modifying InfectStatus to add another case to the switch statement to check for that new entity.

The chosen design was used as it is more extensible. When a new class implementing Infectable is added, the only thing that needs to be done is to define how to handle the infection logic in that class. This does not require modification of any other classes, which adheres to the Open-Closed Principle (OCP). Furthermore, this adheres to the Dependency-Inversion Principle (DIP) since, unlike the alternative design, InfectStatus has a dependency on the Infectable interface rather than each of the concrete classes (ContractedWorker, Undead etc.). This removes the tight coupling between InfectStatus and those concrete classes.

The chosen design here is actually similar to the design around poison and burn effects in Assignment 1, with the same benefits as well.

Decision 1b: The parasite needs to be able to infect adjacent entities.

Chosen design: Create InfectBehaviour and InfectAction. InfectBehaviour will be used by the parasite. InfectBehaviour returns an InfectAction for the parasite to execute.

Advantages	Disadvantages
InfectBehaviour standardizes the logic for an actor infecting other entities. New EclipseActor classes could just have this behaviour added to their behaviours rather than needing to repeat code.	Multiple additional classes are made to handle infecting, increasing complexity of the design.

Alternative design: In the Parasite's playTurn method, before iterating through any of its behaviours, perform the logic related to infecting (looking for an Infectable entity in adjacent locations, giving them InfectStatus and killing the parasite itself) directly.

Advantages	Disadvantages
Avoids adding additional classes, keeping the design simpler.	Any future actors that can infect other entities cannot reuse this. They would have to define the same logic in their playTurn method, which is code duplication.

The chosen design was used as the logic for infecting is defined in one place, InfectBehaviour. Since it might be possible in the future that a new EclipseActor gets added that has the ability to infect others as well, it does not need to define the same logic again, instead just using InfectBehaviour which already exists. This adheres to DRY as there's no code duplication necessary, and promotes code reusability.

Also, unlike the alternative design, instead of having Parasite be responsible for finding Infectables, giving the status effect and dying instantly, this design moves the responsibility of inflicting the status effect to InfectAction* and finding Infectables to InfectBehaviour, demonstrating separation of concerns and adhering to the Single Responsibility Principle (SRP).

*The parasite dying instantly will be caused by InfectAction, but this is part of another design decision.

Note that the behaviour priorities for Parasite (and other EclipseActors) have been declared as class constants (static final), eliminating the magic numbers that were present from the design in Assignment 1. This changes the connascence of meaning into connascence of name (weaker), improving readability and maintainability.

Decision 1c: The parasite needs to instantly die when it successfully infects a target.

The chosen design uses an Infector interface, which requires concrete classes that implement it define the effect of what happens to it when it infects a target. Parasite implements this and the effect is to just die. InfectAction will give the InfectStatus to the target, and then activate this effect to the Infector.

An alternative design would be to have InfectAction kill the Parasite directly, which doesn't need an interface and is simpler to implement.

But consider, hypothetically, a new EclipseActor that can infect others, but doesn't die from infecting (some other effect). The current design is more flexible and extensible as it easily supports this addition without needing to change InfectAction to handle the new effect for that new class. This adheres to the OCP.

Decision 2a: The hole in Deprecated99 and Overflow20 needs to spawn different creatures.

Chosen design: Holes will have an attribute which is a list of what it can spawn (Spawners*). This list is defined and passed in by each GameMap when creating Holes (as part of its map structure).

*This is part of another decision discussed later.

Advantages	Disadvantages
Adding new GameMaps does not require modifying Hole since GameMap defines what their Holes can spawn.	DefaultGroundConstructor no longer works for Holes. GameMaps need to set their Hole grounds manually, which is less convenient.

Alternative design: When spawning, Holes will check which GameMap they're on (this is possible since its tick method provides its location, where it can get its GameMap). A switch statement, instanceof and downcasting is then used to determine the creatures it can spawn.

Advantages	Disadvantages
Still allows the DefaultGroundConstructor to be used for Holes, allowing for easier set up of the GameMap's structure.	Adding new GameMaps require also modifying each Hole, or even other ground spawners like Vents if designed similarly to Hole.

The chosen design was used as it is more extensible. Adding more GameMaps requires no modification to the Hole class. This adheres to the OCP. While the alternative is more convenient to set up new GameMaps, the disadvantages of violating OCP and having less readability and maintainability in Hole outweighs it. Additionally, this design uses dependency injection for the Hole's list of Spawners, which means Hole has a dependency to the Spawner interface rather than to each concrete class implementing Spawner. This demonstrates the DIP.

Decision 2b: Hole and Vent have effects after successfully spawning a creature. Should an interface (e.g. GroundSpawner) be added where this effect is to be defined?

The chosen design doesn't have this interface, simply because it's redundant. These effects are called internally within each respective class and is not used anywhere else. Adding this interface unnecessarily increases complexity and would be a violation of KISS.

Decision 3: Creatures spawned trigger an immediate environmental effect, no matter what spawned them.

Chosen design: Create a Spawner interface and concrete classes (e.g. UndeadSpawner) that implements this. These will handle spawning an actor when given a location, as well as any other behaviours (like those immediate effects).

Advantages	Disadvantages
Any creature spawned using the Spawner will always trigger the immediate effect (if there's one defined for the respective creature).	Additional interface and concrete classes add complexity to the design.
Avoids code duplication on the entity that spawns the creature (no need for each one to check if the creature has an effect).	-

Alternative design:

(Note that this alternative doesn't have a Spawner interface, so Hole/Vent would have a List<Supplier<Actor>> instead of List<Spawner>.)

Add an EnvironmentalTriggerer interface for Undead, Slime and Parasite to implement. Those classes will define the environmental effect. Whenever creatures are spawned, the class spawning them (e.g. Hole) needs to check if the creature implements this interface (via downcasting), and activate the effect if it does.

Advantages	Disadvantages
Does not need an additional interface and additional concrete class for each creature that has the immediate effect.	Everything that spawns creatures must check if the creature implements the interface, then activate the effect. Code duplicating aside, this might be easily missed, leading to logic errors.
-	Involves downcasting, although this uses the engine's code which isn't too much of an issue.

The chosen design was used as each Spawner concrete class guarantees that when it spawns a creature, it will trigger the immediate effect. Anything that spawns creatures do not have to check if the creature they're spawning has an immediate effect or not. Unlike the alternative, this avoids needing code duplication across Hole, Vent, ContractedWorker etc. for doing this check, adhering to DRY. The Spawner concrete classes also shows high cohesion as they group the spawning logic and the immediate effects together, which is more readable and maintainable.